

Day 13

Managing Users and Permissions in Linux

Soft Links (Shortcuts) and Hard Links: explained in simple language

What you will learn	Simple meaning
Why links?	Avoid typing long paths again and again
Soft link	A shortcut that points to the file's path
Hard link	A second real name for the same data
Differences	Inode number, what happens when the original is deleted
Real-life use	How a live server uses links to switch between versions

Tip: This guide is based on your Day 13 class video. Practice in a test folder and use only test files.

1. Simple Terms You Must Know

Term	Meaning in simple words
Link	A name that leads to a file.
Soft link (symbolic link)	A small file that stores the path of another file. Works like a shortcut.
Hard link	One more real name for the same file data.
Original / Target file	The file that the link points to.
Path	The address of a file, like /home/student/docs/my_file.txt.
Inode	The record that stores a file's details and where its data is on the disk. Each file has an inode number.
Link count	How many names point to one inode (2nd column of ls -l).
Broken link	A soft link whose target file is gone. It points to nothing.
Syntax	The correct way to write a command.

2. Why Do We Need Links?

Imagine a file that you use every day, but its path is very long:

```
/home/student/projects/company/reports/2026/october/final/my_file.txt
```

Typing or remembering this long path again and again is tiring. The solution is a **shortcut**.

Easy picture: Windows comparison: a file is stored deep inside the C: drive, in Program Files. You right-click it and create a **shortcut on the Desktop**. Now you open it from the Desktop, but the real file stays where it is. A **soft link** in Linux works the same way.

3. Soft Links (Symbolic Links)

Creating a soft link

Use of ln -s: link with the -s option (soft/symbolic) creates a shortcut.

Syntax: `ln -s target_file link_name`

Part	Meaning
<code>ln</code>	The link command.
<code>-s</code>	Make a soft (symbolic) link.
<code>target_file</code>	The real file (or folder) you want to point to.
<code>link_name</code>	The name of the new shortcut.

```
$ ln -s /home/student/data/my_file.txt my_file_sl
$ ls
my_file_sl
```

Here *sl* is just a name hint for 'soft link'. You can use any name. If you give no link name, Linux creates the link with the same name as the target.

How to recognise a soft link: `ls -ltr`

Use: a long list of files. The options mean: **-l** = long details, **-t** = sort by time, **-r** = reverse order (newest at the bottom).

```
$ ls -ltr
-rw-r--r-- 1 student student 25 Dec 18 10:00 my_file.txt
lrwxrwxrwx 1 student student 28 Dec 18 10:05 my_file_sl -> /home/student/data/my_file.txt
```

What you notice	Meaning
<code>l</code> at the start	The file type is <code>l</code> = link. (A normal file has <code>-</code> and a folder has <code>d</code> .)
<code>-> /home/student/...</code>	The arrow shows exactly which file this shortcut points to.
Different colour	Most terminals show links in a different colour (often cyan). Colours may differ on your system.
<code>rw-rwxrwx</code>	Soft link permissions always look like this. The real permissions come from the target file.
Size 28	The size is only the length of the path text stored inside the link.

Using a soft link: read

```
$ cat my_file_sl
Hello, this is the sample file.
```

You get the same result as `cat` on the original file.

Using a soft link: edit

You can also **edit** the real file through the shortcut:

```
$ echo "This is a new line from shortcut" >> my_file_sl
$ cat my_file_sl
Hello, this is the sample file.
This is a new line from shortcut

$ cat /home/student/data/my_file.txt          # original file
Hello, this is the sample file.
This is a new line from shortcut
```

`>>` adds text at the **end** of a file (while a single `>` replaces everything). The new line also appears in the original, because the shortcut and the original are the same data.

Many links to one file

You can create **more than one** shortcut to the same file, even in different folders:

```
$ ln -s /home/student/data/my_file.txt sl1
$ ln -s /home/student/data/my_file.txt /tmp/sl2
$ ls -ltr
lrwxrwxrwx 1 student student 28 Dec 18 10:10 sl1 -> /home/student/data/my_file.txt
```

Good habit: Always use the **full path** (starting with /) for the target when you plan to move the link around. A short relative path is read from the link's own folder, so the link may break if the link is in a different folder.

Soft links have a different inode number

ls -li shows the inode number (the **i** stands for inode).

```
$ ls -li
765 -rw-r--r-- 1 student student 58 Dec 18 10:00 my_file.txt
763 lrwxrwxrwx 1 student student 28 Dec 18 10:05 my_file_sl -> my_file.txt
```

The original file has inode **765** and the soft link has inode **763**. They are **different**, which shows the soft link is a separate small file that only stores a path. (Numbers in your system will differ.)

Removing a soft link

A soft link is like a shortcut on the desktop. To remove it, just delete it with **rm**. The original file is not affected.

```
$ rm my_file_sl          # removes only the shortcut
$ ls
my_file.txt              # original is safe
```

Careful: Do not put a slash at the end when you delete a link to a folder (use **rm linkname**, not **rm linkname/**).

What happens when the original is deleted? Broken link

```
$ rm my_file.txt          # delete the ORIGINAL
$ ls -l
lrwxrwxrwx 1 student student 28 Dec 18 10:05 my_file_sl -> my_file.txt (shown in red)

$ cat my_file_sl
cat: my_file_sl: No such file or directory
```

The shortcut still exists and still shows the old path, but there is nothing at that path. This is a **broken link** (also called a dangling link). Most terminals show it in **red**. If you see red, think: broken link.

Small correction: Your class notes say 'the link will be removed if the original file is deleted'. More exactly: the link is **not** removed. It stays, but it becomes **broken** and useless until you restore the file or delete the link.

```
# If you recreate the original file with the same name, the link works again
$ echo "back again" > my_file.txt
$ cat my_file_sl
back again
```

4. Hard Links

A hard link is another **real name** for the same data. It looks like a normal file, not a shortcut.

Syntax: `ln original_file hard_link_name` (just **ln**, with **no -s**)

```
$ touch my_file.txt           # create an empty file
$ ln my_file.txt my_file_hl   # make a hard link
$ ls
my_file.txt  my_file_hl

$ ls -ltr
-rw-r--r-- 2 student student 0 Dec 18 11:00 my_file.txt
-rw-r--r-- 2 student student 0 Dec 18 11:00 my_file_hl
```

What you notice	Meaning
Starts with - (not l)	It looks like a normal file. There is no arrow.
Link count is 2	Two names point to the same inode.

Same inode number

```
$ ls -li
763 -rw-r--r-- 2 student student 0 Dec 18 11:00 my_file.txt
763 -rw-r--r-- 2 student student 0 Dec 18 11:00 my_file_hl
```

Both names show inode **763**. This means both names lead to the **same data on the disk**.

Changing one changes both

```
$ echo "This is sample data for hard link" > my_file_hl
$ cat my_file_hl
This is sample data for hard link
$ cat my_file.txt
This is sample data for hard link
```

The big advantage: delete the original, data stays

```
$ rm my_file.txt # delete the original name
$ ls -l
-rw-r--r-- 1 student student 34 Dec 18 11:05 my_file_hl

$ cat my_file_hl
This is sample data for hard link # still works!
```

The hard link still works and the link count goes down from 2 to 1. There is **no broken link**, because a hard link is not a pointer to a path. It is a full name of the data.

Why does the data stay? (Interview question)

- Data lives on the disk. The **inode** is the record that knows where that data is.
- Every file name points to an inode. A hard link creates a **second name** that points to the **same inode**.
- Deleting one name only removes that name and reduces the link count by 1.
- The data is erased from the disk only when the link count becomes **0** (no names left).
- A soft link points to the **name** (path), not the inode. When that name is deleted, the soft link has nothing to find.

Small correction: Your class notes describe the inode number as a disk block. More exactly: the inode is a record that **points to** the disk blocks where the data is stored. The idea that both names reach the same data is correct.

5. Hard Link vs Soft Link

Point	Soft link (ln -s)	Hard link (ln)
Looks like	Shortcut (type l, with an arrow)	Normal file (type -)
Inode number	Different from the original	Same as the original
Stores	The path of the target	Nothing extra, just another name for the data
Original deleted	Link becomes broken (red)	Link still works , data is safe
Link deleted	Original not affected	Original not affected (link count reduces)
Edit through the link	Changes the original	Changes the same data
Link to a folder	Allowed	Not allowed
Across file systems	Allowed	Not allowed
Create with	ln -s original link	ln original link
Remove with	rm link	rm link

Remember: Short memory trick: Soft = shortcut (breaks if the original is deleted). Hard = second name (survives).

6. Real-Life Use Case: Switching Application Versions

A server application reads a configuration file with a **fixed name**, for example `config.xml`. New versions of the file are made often (bug fixes, new features, client changes): `config_v1.xml`, `config_v2.xml`, `config_v3.xml`.

Without links: you rename v2 to `config.xml`. Only one file can have that name, and later nobody can tell at a glance which version is in use.

With a soft link: keep all versions, and make **config.xml** a shortcut to the version you want.

```
$ ln -s config_v2.xml config.xml
$ ls -l config.xml
lrwxrwxrwx 1 app app 13 Dec 18 12:00 config.xml -> config_v2.xml
```

You can see the active version instantly from the arrow. When version 3 arrives, remove the old link and make a new one:

```
$ rm config.xml
$ ln -s config_v3.xml config.xml

# Or in one step (-f = force, replace the old link):
$ ln -sf config_v3.xml config.xml
```

Benefit	Meaning
Clear view	The arrow shows the version currently in use.
No renaming	All versions are kept safely as separate files.
Easy rollback	If v3 has a problem, point the link back to v2.
No application change	The app always reads <code>config.xml</code> and does not know about the switch.

The same idea is used with Java **.jar** files, **/etc/alternatives** (for example the 'java' command), and web server settings.

7. Useful Extra Commands

Command	Use
readlink linkname	Shows which path a soft link points to.
ls -l linkname	Shows the arrow and target.
stat filename	Shows inode number, link count and type in full detail.
find . -xtype l	Finds broken soft links in the current folder.
ln -sf target link	Create a soft link and replace the old one if it exists.
unlink linkname	Removes a link (same effect as <code>rm</code> for one link).

8. Quick Revision Cheat Sheet

Command	What it does
<code>ln -s target link</code>	Create a soft link (shortcut)
<code>ln original link</code>	Create a hard link
<code>ls -ltr</code>	Long list, sorted by time, newest at the end
<code>ls -li</code>	Show inode numbers
<code>rm link</code>	Remove a link
<code>readlink link</code>	Show target of a soft link
<code>find . -xtype l</code>	Find broken soft links
<code>ln -sf target link</code>	Replace an existing soft link

9. Practice Lab (Do in Order)

1. `mkdir linklab && cd linklab` (practice folder)
2. `echo "Hello" > my_file.txt` (create the original)
3. `ln -s my_file.txt my_file_sl` (soft link)
4. `ls -li` (soft link has a different inode and an arrow)
5. `echo "new line" >> my_file_sl` (edit through the link)
6. `cat my_file.txt` (the original has the new line too)
7. `ln my_file.txt my_file_hl` (hard link)
8. `ls -li` (`my_file.txt` and `my_file_hl` have the same inode, count 2)
9. `rm my_file.txt` (delete the original)
10. `ls -l` (soft link is broken and red)
11. `cat my_file_sl` (error: No such file or directory)
12. `cat my_file_hl` (still works)
13. `rm my_file_sl` (remove the broken soft link)
14. `cd .. && rm -r linklab` (clean up)

Practice and interview questions

- What is a soft link? Which command creates it?
- How can you recognise a soft link in the `ls -l` output?
- What happens to a soft link when the original file is deleted? What colour does it show?
- What happens to a hard link when the original file is deleted? Why?
- How do the inode numbers of a soft link and a hard link compare with the original?
- Does deleting a soft link affect the original file?
- Give one real-life use of soft links on a server.
- Why can hard links not point to folders or cross file systems? (Hint: they share an inode, which exists only inside one file system.)

Summary: Conclusion: Soft links are shortcuts that point to a path, and they break if the original is deleted. Hard links are extra names for the same data, and they survive. Knowing this helps you manage live servers safely.